Modern adatbázis rendszerek

JSON és JSON Schema elmélet

Kovács Dániel
F9Y7TW

2026. április 21.

Mi az a JSON?

JavaScript Object Notation

- Könnyű, szöveges adatcsere formátum.
- Nyelvfüggetlen, de JavaScript alapú.
- Ember és gép számára is könnyen olvasható.

JSON vs. XML összehasonlítás

Szempont	JSON	XML
Olvashatóság	Letisztult, kulcs-érték párok.	Tag-alapú, beszédesebb.
Adatméret	Kisebb (nincsenek záró tagek).	Nagyobb (redundáns tagek).
Típusosság	Támogat típusokat (szám, logikai, tömb).	Alapvetően minden szöveg.
Feldolgozás	Gyorsabb, natív JS támogatás.	Lassabb, DOM fa építése szükséges.
Szerkezet	Objektum-orientált szemlélet.	Dokumentum-központú.

Megjegyzés: Webes adatcserére a JSON az iparági lett sztenderd, míg az XML bonyolultabb dokumentum-struktúrákhoz előnyösebb, habár már itt is elkezd megjelenni.

Konverziós szabályok

Hogyan alakítsuk át az XML-t JSON-re?

- **Komplex tipusok:** Objektumokká vállnak {...}
- **Attribútumok:** Mezőkké válnak (*ekod* → `"_ekod"`).
- **Lista típusok:** Az ismétlődő elemek tömbbe kerülnek (`[]`).

```
{
  "etterem": [
    {
      "_ekod": "e1",
      "nev": "Trófea",
      "cim": {
        "varos": "Budapest",
        "utca": "Visegrádi"
      }
    }
  ]
}
```

A JSON Schema felépítése

Alapvető meta-adatok:

- `$schema`: A használt szabvány verziója (pl. draft-07).
- `title` / `description`: Dokumentációs célú megjegyzések.

Strukturális elemek:

- `type`: Meghatározza az adat típusát (*object*, *array*, *string*, *number*, *boolean*, *null*).
- `properties`: Itt definiáljuk a JSON objektum kulcsait és azok elvárt típusait.
- `required`: Egy lista, amely tartalmazza a kötelezően megadandó mezők neveit.

Validációs megkötések:

- **Számoknál:** `minimum`, `maximum`.
- **Szövegeknél:** `minLength`, `maxLength`, `pattern` (RegEx).
- **Tömböknél:** `minItems`, `uniqueItems`.

JSON és Schema összehasonlítása

JSON adat:

```
{
  "nev": "Béla",
  "kor": 500,
  "gyerekek": [
    "Pista",
    "Józsi"
  ]
}
```

JSON Schema:

```
{
  "type": "object",
  "properties": {
    "nev": {"type": "string"},
    "kor": {
      "type": "integer",
      "minimum": 0
    },
    "gyerekek": {
      "type": "array",
      "items": {"type": "string"}
    }
  },
  "required": ["nev", "kor"]
}
```

JSON Validálása és Ellenőrzése

A folyamat lépései:

1. **Séma definiálása:** Meghatározzuk a várt struktúrát JSON formátumban.
2. **Beolvasás:** A Jackson könyvtár segítségével betöltjük az adatot a memóriába.
3. **Összevetés:** A validátor összehasonlítja a JSON fájlt a sémával.

JSON Validáció logikája Java-ban

```
ObjectMapper mapper = new ObjectMapper();
JsonNode data = mapper.readTree(new File("JSON.json"));
JsonNode schemaNode = mapper.readTree(
new File("schema.json"));

JsonSchemaFactory factory = JsonSchemaFactory.getInstance(
SpecVersion.VersionFlag.V4);
JsonSchema schema = factory.getSchema(schemaNode);
Set<ValidationMessage> errors = schema.validate(data);

if (errors.isEmpty()) {
    System.out.println("JSON_is_good");
} else {
    errors.forEach(err -> System.out.println(
    "Hibak:" + err.getMessage()));
}
```

Megjegyzés: Online validátort is lehet hazsnálni:
https://www.jsonschemavalidator.net/ (biztonsági problémák)

JSON kezelése Java-ban

A két legfontosabb csomópont típus:

- **JsonNode (Read-only):** Alapértelmezett típus beolvasáskor. Csak olvasásra szolgál, az adatok kinyerésére optimalizált (pl. `get()`, `asText()`).
- **ObjectNode (Mutable):** A `JsonNode` leszármazottja, amely lehetővé teszi az adatok módosítását. Ahhoz, hogy szerkeszteni tudjunk, a beolvasott csomópontot `ObjectNode`-dá kell castolni.

Műveletek típusai:

- **Keresés:** `get()` vagy `path()` metódusokkal.
- **Manipuláció:** Új mezők hozzáadása, meglévők frissítése vagy törlése.
- **Kiírás:** A módosított fa visszaírható JSON fájlba az `ObjectMapper` segítségével, vagy újat csinálhatunk(10-11 dia).

JSON kezelés: Gyakorlati példa

```
JsonNode foszakacsok = root.get("vendeglatas").
get("foszakacs");
for (JsonNode szakacs : foszakacsok) {
    String nev = szakacs.get("nev").asText();
    String kor = szakacs.get("eletkor").asText();
    String iskola = szakacs.get("vegzettseg").asText();

    System.out.println("Name: " + nev + " |
    age: " + kor + " | qualifications: " + iskola);
}
```

JSON létrehozása Java-ban

Szükséges elemek:

- `ObjectMapper`: A JSON generálás
- `ObjectNode`: Az új JSON objektum ({ }). Létrehozás: `mapper.createObjectNode()`.
- `ArrayNode`: A JSON tömb ([ ]). Létrehozás: `mapper.createArrayNode()`.

Alapműveletek:

- `put("kulcs", ertek)`: Adat (szöveg, szám, logikai) beírása egy `ObjectNode`-ba.
- `add(ertek)`: Elem hozzáfűzése egy `ArrayNode`-hoz.
- `writeValue(file, fa)`: A kész adatstruktúra kiírása a merevlemezre.

JSON készítése Java-ban

Új JSON adatszerkezet építése memóriában:

```
ObjectMapper mapper = new ObjectMapper();

ArrayNode szakacsLista = mapper.createArrayNode();

ObjectNode ujSzakacs = mapper.createObjectNode();
ujSzakacs.put("nev", "Peter␣Smith");
ujSzakacs.put("eletkor", 28);
ujSzakacs.put("reszleg", "Saucier");

szakacsLista.add(ujSzakacs);

mapper.writerWithDefaultPrettyPrinter().writeValue(
new File("uj_fajl.json"), ujSzakacs);
vagy
mapper.writeValue(new File("uj_fajl.json"), lista);
```

Vége

Köszönöm a figyelmet!

És irány a gyakorlat!